Artificial Intelligence course

6th Semester

Bachelor in Informatics and Computer Engineering

Heuristic Search and the A* Algorithm

Nuno Leite

28 April 2022

best-first (A^*) algorithm

- Graph searching in problem solving typically leads to the problem of combinatorial complexity due to the proliferation of alternatives
  - Heuristic search aspires to fight this problem efficiently
  - One way of using heuristic information about a problem is to compute numerical heuristic estimates for the nodes in the state space

best-first (A^*) algorithm

- Such an estimate of a node indicates how promising a node is with respect to reaching a goal node
- The idea is to continue the search always from the most promising node in the candidate set – best-first search

best-first (A^*) algorithm

- A best-first search program can be derived as a refinement of a breadth-first search program
- We will from now on assume that a cost function is defined for the arcs of the state space
  - So $c(n, n')$ is the cost of moving from a node n to its successor n' in the state space

best-first (A^*) algorithm

- Let the heuristic estimator be a function f, such that for each node n of the space, $f(n)$ estimates the 'difficulty' of n
  - Accordingly, the most promising current candidate node is the one that minimizes f
- $f(n)$ will be constructed so as to estimate the cost of a best solution path from the start node, s, to a goal node, under the constraint that this path goes through n (Figure 12.1): $f(n) = g(n) + h(n)$

best-first (A^*) algorithm

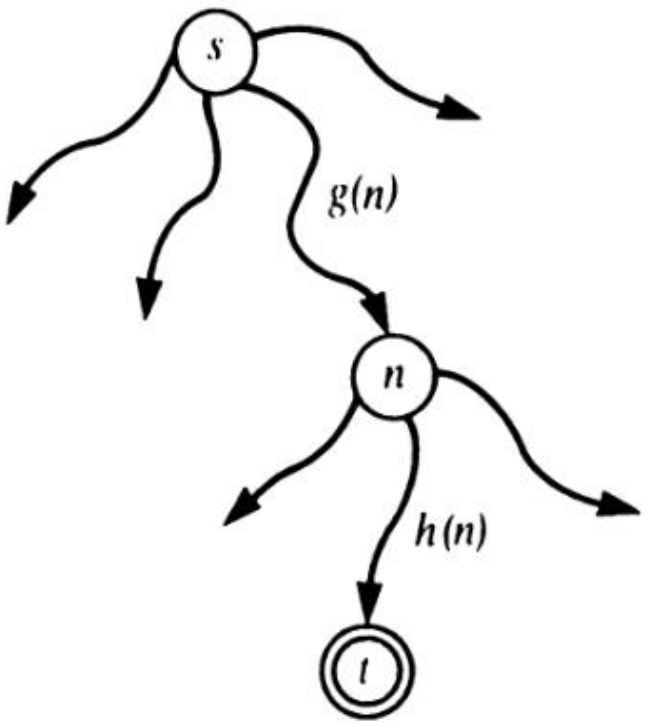


Figure 12.1 Construction of a heuristic estimate $f(n)$ of the cost of the cheapest path from s to t via n: $f(n) = g(n) + h(n)$.

best-first (A^*) algorithm

- $f(n) = g(n) + h(n)$
- $g(n)$ is an estimate of the cost of an optimal path from s to n
  - can be computed as the sum of the arc costs on the path
  - This path is not necessarily an optimal path from s to n (there may be a better path from s to n, not yet found by the search)

best-first (A^*) algorithm

- $h(n)$ is an estimate of the cost of an optimal path from n to t
  - $h(n)$ is typically a heuristic guess, based on the algorithm's general knowledge about the particular problem
  - As h depends on the problem domain there is no universal method for constructing h

best-first (A^*) algorithm

- In best-first search, the search process consists of a number of competing subprocesses, each of them exploring its own alternative
  - that is, exploring its own subtree
- Among all these competing processes, only one is active at each time:
  - the one that deals with the currently most promising alternative; that is, the alternative with the lowest f-value

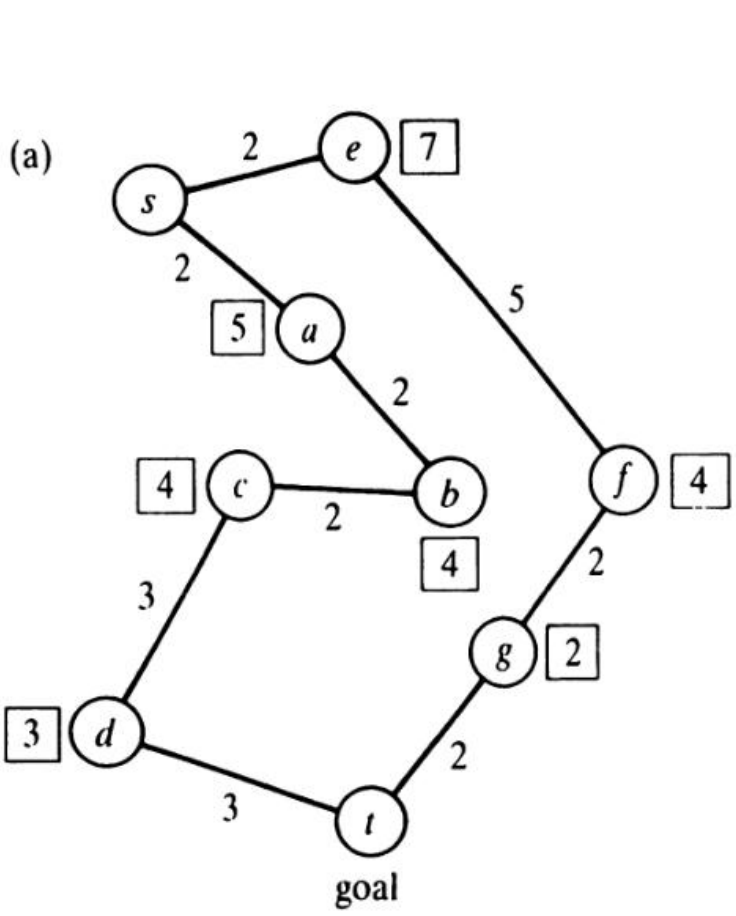
best-first (A^*) algorithm

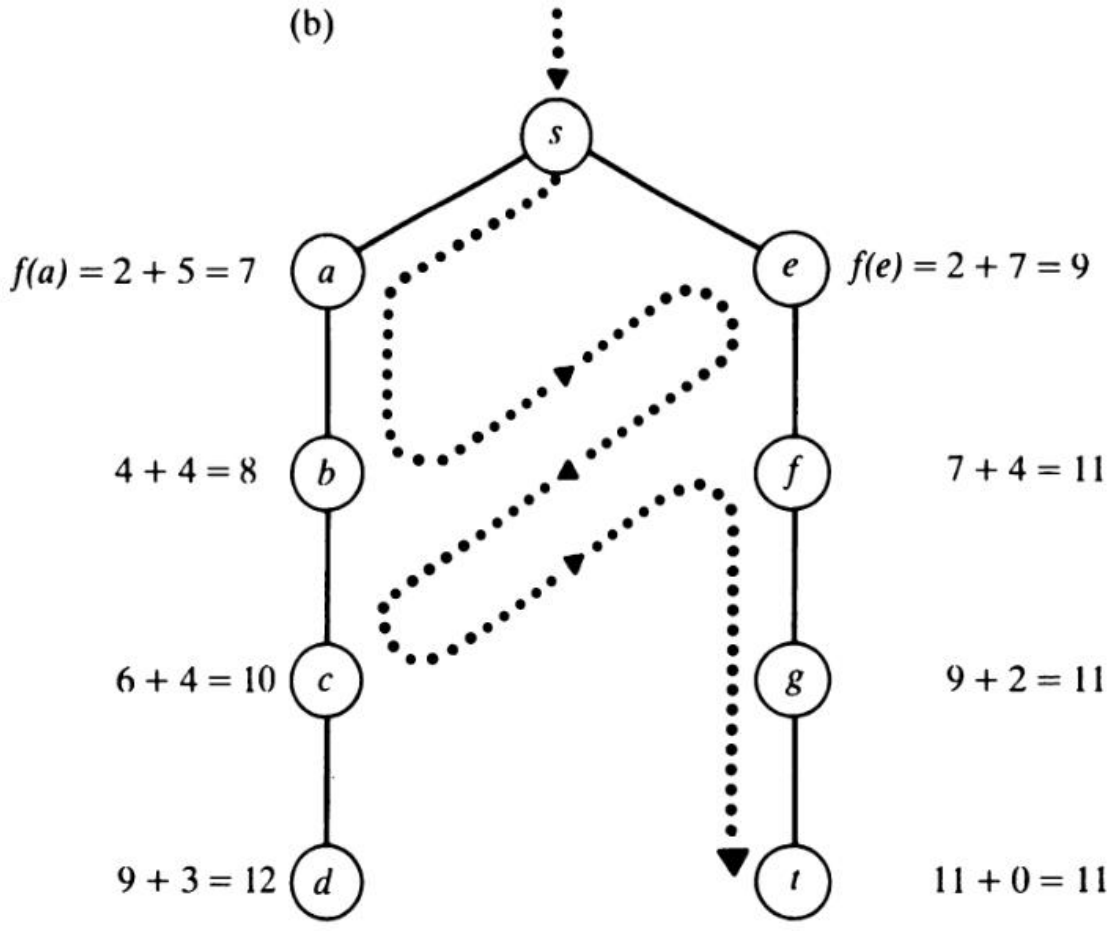
- The remaining processes have to wait until the current f-estimates change so that some other alternative becomes more promising
  - Then the activity is switched to this alternative

best-first (A^*) algorithm

- We can imagine this activate-deactivate mechanism to work as follows:
  - the process working on the currently top-priority alternative is given some budget and the process is active until this budget is exhausted
  - During this activity, the process keeps expanding its subtree and reports a solution if a goal node was encountered
  - The budget for this run is defined by the heuristic estimate of the closest competing alternative

best-first (A^*) algorithm





best-first (A^*) algorithm

Figure 12.2 Finding the shortest route from s to t in a map. (a) The map with links labelled by their lengths; the numbers in the boxes are straight-line distances to t. (b) The order in which the map is explored by a best-first search. Heuristic estimates are based on straight-line distances. The dotted line indicates the switching of activity between alternate paths. The line shows the ordering of nodes according to their f-values; that is, the order in which the nodes are *expanded* (not the order in which they are generated).

- For a given city (node X), we use as $h(X)$ the straight-line distance to the goal node, t, denoted by $dist(X, t)$. So: $f(X) = g(X) + h(X) = g(X) + dist(X, t)$

best-first (A^*) algorithm

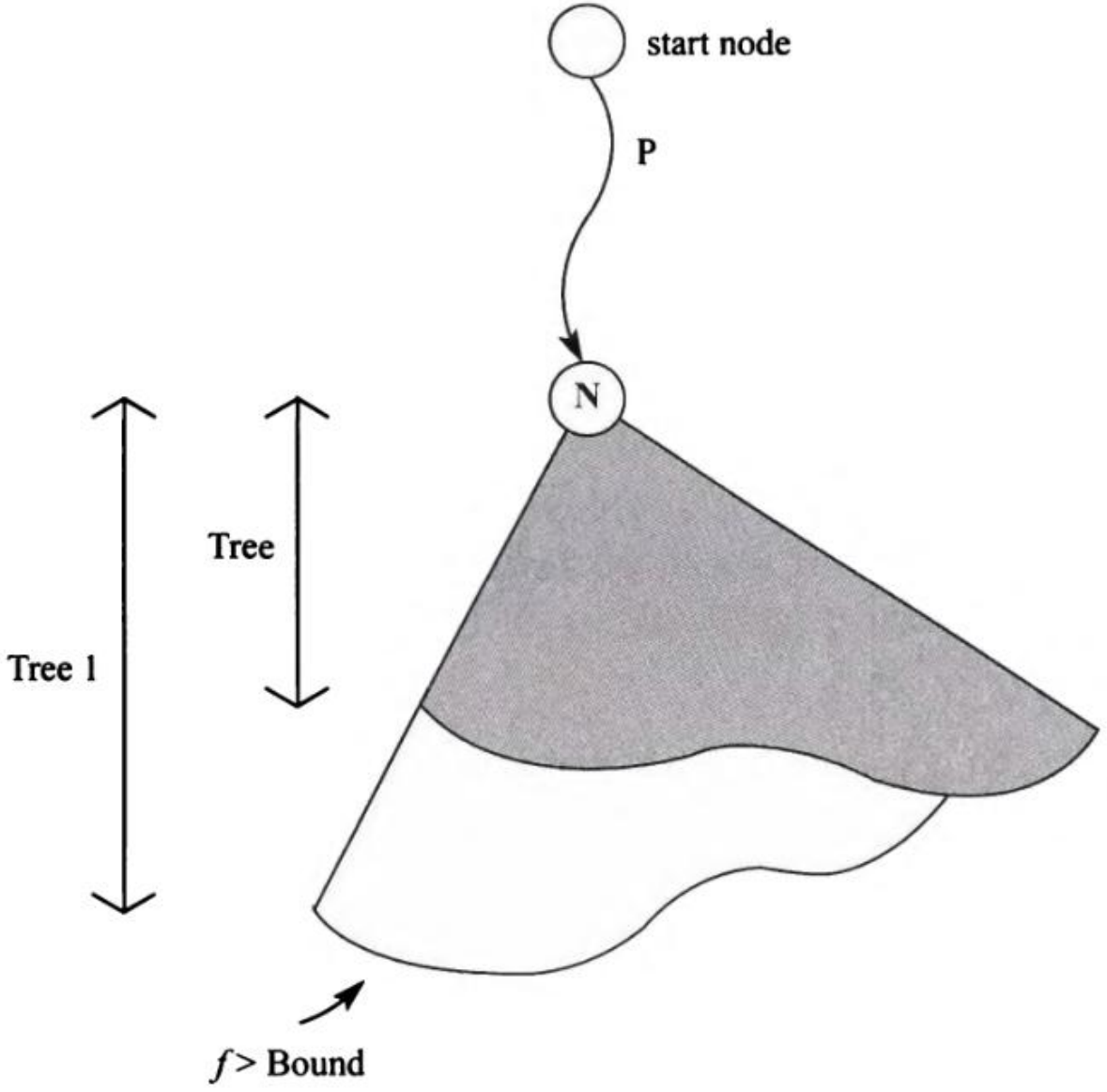
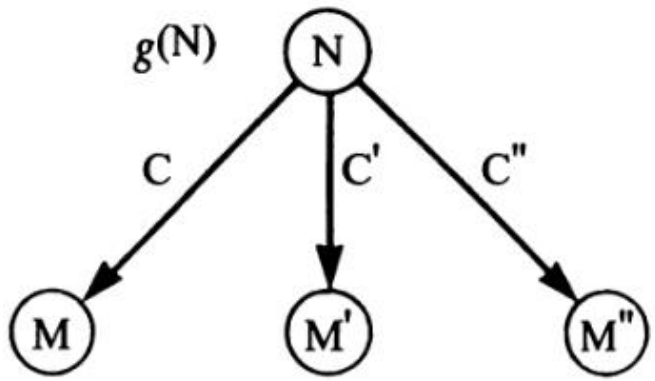


Figure 12.4 The **expand** relation: expanding **Tree** until the f-value exceeds **Bound** results in **Tree1**.

best-first (A^*) algorithm



$$g(M) = g(N) + C$$
$$f(M) = g(M) + h(M)$$

Figure 12.5 Relation between the g-value of node N, and the f- and g-values of its children in the search space.

Admissible heuristic function

An important result from the mathematical analysis of A^* is:

A search algorithm is said to be *admissible* if it always produces an optimal solution (that is, a minimum-cost path) provided that a solution exists at all. Our implementation, which produces all solutions through backtracking, can be considered admissible if the *first* solution found is optimal. Let, for each node n in the state space, $h^*(n)$ denote the cost of an optimal path from n to a goal node. A theorem about the admissibility of A* says: an A* algorithm that uses a heuristic function h such that for all nodes n in the state space

$$h(n) \leq h^*(n)$$

is admissible.

Admissible heuristic function

- Even if we do not know the exact value of h^* we just have to find a lower bound of h^* and use it as h in A^*
  - This is sufficient guarantee that A^* will produce an optimal solution

Admissible heuristic function

- There is a trivial lower bound, namely: $h(n) = 0$, for all n in the state space
  - This indeed guarantees admissibility
  - The disadvantage of $h = 0$ is, however, that it has no heuristic power and does not provide any guidance for the search
  - A^* using $h = 0$ behaves similarly to the breadth-first search

best-first (A^*) algorithm

See pages 282-289 of Ivan Bratko's Prolog Book